www.android.universityupdates.in | www.universityupdates.in | www.ios.universityupdates.in

UNIT-II

ASSOCIATION RULES

2.1 Introduction:

- ARM is to find out association rules or Frequent patterns or subsequences or correlation relationships among large set of data items that satisfy the predefined minimum support and confidence from a given database.

- **Frequent patterns** are patterns (such as itemsets, subsequences, or substructures) that appear in a data set frequently. For example, a set of items, such as milk and bread, that appear frequently together in a transaction data set is a *frequent itemset*. A subsequence, such as buying first a PC, then a digital camera, and then a memory card, if it occurs frequently in a shopping history database, is a (*frequent*) *sequential pattern*. A *substructure* can refer to different structural forms, such as subgraphs, subtrees, or sublattices, which may be combined with itemsets or subsequences.

What Is Association Mining?

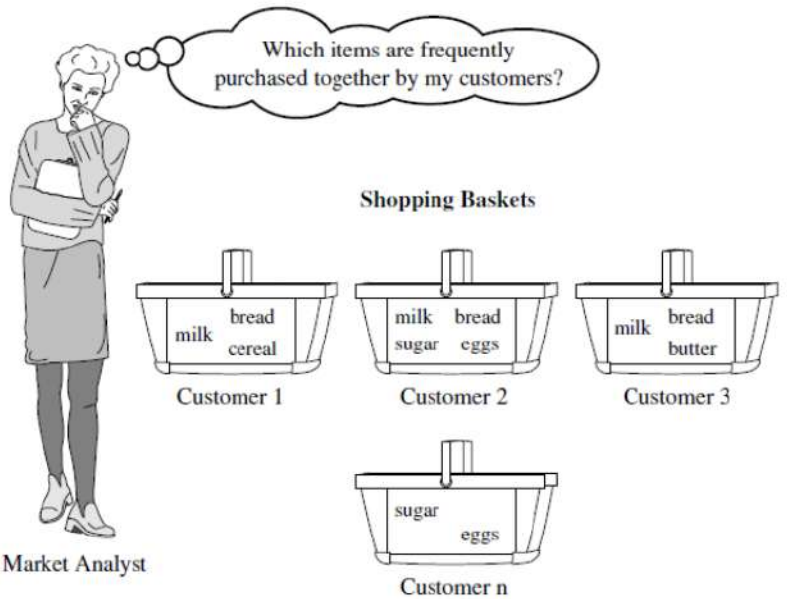
- **Association rule mining:**
  - Finding frequent patterns, associations, correlations, or causal structures among sets of items or objects in transaction databases, relational databases, and other information repositories.
- **Applications:**
  - Basket data analysis, cross-marketing, catalog design, loss-leader analysis, clustering, classification, etc.

2.1.1 Market Basket Analysis:

Frequent itemset mining leads to the discovery of associations and correlations among items in large transactional or relational data sets. A typical example of frequent itemset mining is market basket analysis. This process analyzes customer buying habits by finding associations between the different items that customers place in their "shopping baskets". The discovery of such associations can help retailers develop marketing strategies by gaining insight into which items are frequently purchased together by customers.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | www.ios.previousquestionpapers.com

www.android.universityupdates.in | www.universityupdates.in | www.ios.universityupdates.in



2.1.2 ***Frequent Itemsets, Closed Itemsets, and Association Rules***

Let I ={*I*1, *I*2, ... , *Im*} be a set of items. Let *D*, the task-relevant data, be a set of database transactions where each transaction *T* is a set of items such that *T* _ I . Each transaction is associated with an identifier, called TID.

An association rule is an implication of the form $A \Rightarrow B$, where $A \subset$ I , $B \subset$ I , and $A \cap B$=f. The rule $A \Rightarrow B$ holds in the transaction set *D* with support *s*, where *s* is the percentage of transactions in *D* that contain $A \subset B$ (i.e., the *union* of sets *A* and *B*, or say, both *A* and *B*). The rule $A \Rightarrow B$ has confidence *c* in the transaction set *D*, where *c* is the percentage of transactions in *D* containing *A* that also contain *B*. This is taken to be the conditional probability, *P*(*B*|*A*). That is,

$$\begin{aligned} support(A \Rightarrow B) &= P(A \cup B) \\ confidence(A \Rightarrow B) &= P(B|A). \end{aligned}$$

If the relative support of an itemset *I* satisfies a prespecified minimum support threshold (i.e., the absolute support of *I* satisfies the corresponding minimum support count threshold), then *I* is a frequent itemset. The set of frequent *k*-itemsets is commonly denoted by *Lk*.

$$confidence(A \Rightarrow B) = P(B|A) = \frac{support(A \cup B)}{support(A)} = \frac{support_count(A \cup B)}{support_count(A)}.$$

www.android.previousquestionpapers.com | www.previousquestionpapers.com | www.ios.previousquestionpapers.com

www.android.universityupdates.in | www.universityupdates.in | www.ios.universityupdates.in

Support is used to eliminate uninteresting rules. Low support is likely to be uninteresting from a business perspective because it may not be profitable to promote items.

Confidence measures the reliability of the inference made by a rule. For rule A→B, the higher confidence , the more likely it is for **B** to be present in transactions that contain **A**. It is used to estimate conditional probability of **B** given **A.**

In general, association rule mining can be viewed as a two-step process:

1. Find all frequent itemsets: By definition, each of these itemsets will occur at least as frequently as a predetermined minimum support count, *min sup*.
2. Generate strong association rules from the frequent itemsets: By definition, these rules must satisfy minimum support and minimum confidence.

2.2 The Apriori Algorithm:

Apriori is a seminal algorithm proposed by R. Agrawal and R. Srikant in 1994 for mining frequent itemsets for Boolean association rules. Apriori employs an iterative approach known as a *level-wise* search, where *k*-itemsets are usedtoexplore (*k*+1)-itemsets. First, the setof frequent 1-itemsets is found by scanning the database to accumulate the count for each item, and collecting those items that satisfy minimum support. The resulting set is denoted *L*1.Next, *L*1 is used to find *L*2, the set of frequent 2-itemsets, which is used to find *L*3, and so on, until no more frequent *k*-itemsets can be found. The finding of each *Lk* requires one full scan of the database.

Apriori Property:

1. *It makes use of "Upward Closure property" (Any superset of infrequent itemset is also an infrequent set). It follows Bottom-up search, moving upward level-wise in the lattice.*
2. *It makes use of "downward closure property"(any subset of a frequent itemset is a frequent itemset).*
3. *If* Support of an itemset exceeds the support of its subsets, then it is known as the anti-monotone property of support.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | www.ios.previousquestionpapers.com

www.android.universityupdates.in | www.universityupdates.in | www.ios.universityupdates.in

2.2.1 Steps in candidate generation:

- Join Step: To find *Lk*, a set of candidate *k*-itemsets is generated by joining *Lk*-1 with itself.
- Prune Step: Any k-itemset that is not frequent cannot be a subset of a frequent (k+1)-itemset .

Table 5.1 Transactional data for an *AllElectronics* branch.

TID	List of item_IDs
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

Based on the *AllElectronics* transaction database,*D*, of Table 5.1

2.2.2 Apriori algorithm steps for finding frequent itemsets in D:-

1. In the first iteration of the algorithm, each item is a member of the set of candidate 1-itemsets, *C*1. The algorithm simply scans all of the transactions in order to count the number of occurrences of each item.
2. Suppose that the minimum support count required is 2, that is, *min sup* = 2. The set of frequent 1-itemsets, *L*1, can then be determined. It consists of the candidate 1-itemsets satisfying minimum support.
3. To discover the set of frequent 2-itemsets, *L*2, the algorithm uses the join $L1 \otimes L1$ to generate a candidate set of 2-itemsets, *C*2.
4. Next, the transactions in*D*are scanned and the support count of each candidate itemset in*C*2 is accumulated, as shown in the middle table of the second row in Figure 5.2.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | www.ios.previousquestionpapers.com

www.android.universityupdates.in | www.universityupdates.in | www.ios.universityupdates.in

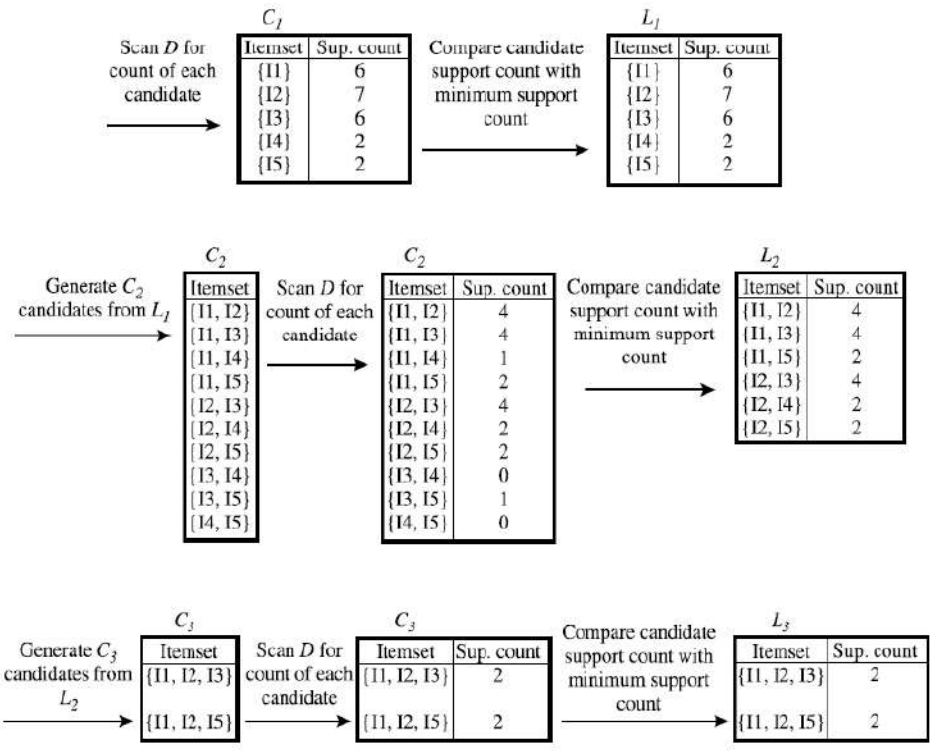


Figure 5.2 Generation of candidate itemsets and frequent itemsets, where the minimum support count is 2.

5. The set of frequent 2-itemsets, $L2$, is then determined, consisting of those candidate 2-itemsets in $C2$ having minimum support.
6. The generation of the set of candidate 3-itemsets,$C3$, is detailed in Figure 5.3.

(a) Join: $C_3 = L_2 \bowtie L_2 = \{\{I1, I2\}, \{I1, I3\}, \{I1, I5\}, \{I2, I3\}, \{I2, I4\}, \{I2, I5\}\} \bowtie \{\{I1, I2\}, \{I1, I3\}, \{I1, I5\}, \{I2, I3\}, \{I2, I4\}, \{I2, I5\}\}$
$= \{\{I1, I2, I3\}, \{I1, I2, I5\}, \{I1, I3, I5\}, \{I2, I3, I4\}, \{I2, I3, I5\}, \{I2, I4, I5\}\}.$

(b) Prune using the Apriori property: All nonempty subsets of a frequent itemset must also be frequent. Do any of the candidates have a subset that is not frequent?

- The 2-item subsets of {I1, I2, I3} are {I1, I2}, {I1, I3}, and {I2, I3}. All 2-item subsets of {I1, I2, I3} are members of L_2. Therefore, keep {I1, I2, I3} in C_3.
- The 2-item subsets of {I1, I2, I5} are {I1, I2}, {I1, I5}, and {I2, I5}. All 2-item subsets of {I1, I2, I5} are members of L_2. Therefore, keep {I1, I2, I5} in C_3.
- The 2-item subsets of {I1, I3, I5} are {I1, I3}, {I1, I5}, and {I3, I5}. {I3, I5} is not a member of L_2, and so it is not frequent. Therefore, remove {I1, I3, I5} from C_3.
- The 2-item subsets of {I2, I3, I4} are {I2, I3}, {I2, I4}, and {I3, I4}. {I3, I4} is not a member of L_2, and so it is not frequent. Therefore, remove {I2, I3, I4} from C_3.
- The 2-item subsets of {I2, I3, I5} are {I2, I3}, {I2, I5}, and {I3, I5}. {I3, I5} is not a member of L_2, and so it is not frequent. Therefore, remove {I2, I3, I5} from C_3.
- The 2-item subsets of {I2, I4, I5} are {I2, I4}, {I2, I5}, and {I4, I5}. {I4, I5} is not a member of L_2, and so it is not frequent. Therefore, remove {I2, I4, I5} from C_3.

(c) Therefore, $C_3 = \{\{I1, I2, I3\}, \{I1, I2, I5\}\}$ after pruning.

Figure 5.3 Generation and pruning of candidate 3-itemsets, C_3, from L_2 using the Apriori property.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | www.ios.previousquestionpapers.com

www.android.universityupdates.in | www.universityupdates.in | www.ios.universityupdates.in

7. The transactions in D are scanned in order to determine $L3$, consisting of those candidate 3-itemsets in $C3$ having minimum support (Figure 5.2).

8. The algorithm uses $L3 \otimes L3$ to generate a candidate set of 4-itemsets, $C4$. Although the join results in {I1, I2, I3, I5}, this itemset is pruned because its subset {I2, I3,I5} is not frequent. Thus, $C4$ = f, and the algorithm terminates, having found all of the frequent itemsets.

2.2.3 pseudo-code for the Apriori algorithm:-

Algorithm: Apriori. Find frequent itemsets using an iterative level-wise approach based on candidate generation.

Input:

- D, a database of transactions;
- *min_sup*, the minimum support count threshold.

Output: L, frequent itemsets in D.

Method:

```
(1)  L1 = find_frequent_1-itemsets(D);
(2)  for (k = 2; Lk-1 ≠ ϕ; k++) {
(3)      Ck = apriori_gen(Lk-1);
(4)      for each transaction t ∈ D { // scan D for counts
(5)          Ct = subset(Ck, t); // get the subsets of t that are candidates
(6)          for each candidate c ∈ Ct
(7)              c.count++;
(8)      }
(9)      Lk = {c ∈ Ck | c.count ≥ min_sup}
(10) }
(11) return L = ∪k Lk;

procedure apriori_gen(Lk-1:frequent (k-1)-itemsets)
(1)  for each itemset l1 ∈ Lk-1
(2)      for each itemset l2 ∈ Lk-1
(3)          if (l1[1] = l2[1]) ∧ (l1[2] = l2[2]) ∧ ... ∧ (l1[k-2] = l2[k-2]) ∧ (l1[k-1] < l2[k-1]) then {
(4)              c = l1 ⋈ l2; // join step: generate candidates
(5)              if has_infrequent_subset(c, Lk-1) then
(6)                  delete c; // prune step: remove unfruitful candidate
(7)              else add c to Ck;
(8)          }
(9)  return Ck;

procedure has_infrequent_subset(c: candidate k-itemset;
          Lk-1: frequent (k-1)-itemsets); // use prior knowledge
(1)  for each (k-1)-subset s of c
(2)      if s ∉ Lk-1 then
(3)          return TRUE;
(4)  return FALSE;
```


www.android.previousquestionpapers.com | www.previousquestionpapers.com | www.ios.previousquestionpapers.com

www.android.universityupdates.in | www.universityupdates.in | www.ios.universityupdates.in

2.2.4 How to Generate Candidates?

- Suppose the items in L_{k-1} are listed in an order
- Step 1: self-joining L_{k-1}

insert into C_k
select $p.item_1, p.item_2, \ldots, p.item_{k-1}, q.item_{k-1}$
from $L_{k-1}\ p, L_{k-1}\ q$
where $p.item_1=q.item_1, \ldots, p.item_{k-2}=q.item_{k-2}, p.item_{k-1} < q.item_{k-1}$

- Step 2: pruning

forall *itemsets c in* C_k do
forall *(k-1)-subsets s of c* do
if *(s is not in* L_{k-1}*)* **then delete** *c* **from** C_k

2.2.5 How to Count Supports of Candidates?

- Why counting supports of candidates a problem?
  - The total number of candidates can be very huge
  - One transaction may contain many candidates
- Method:
  - Candidate itemsets are stored in a *hash-tree*
  - *Leaf* node of hash-tree contains a list of itemsets and counts
  - *Interior* node contains a hash table
  - *Subset function*: finds all the candidates contained in a transaction

Example of Generating Candidates

- L_3={*abc, abd, acd, ace, bcd*}
- Self-joining: L_3*L_3
  - *abcd* from *abc* and *abd*
  - *acde* from *acd* and *ace*
- Pruning:
  - *acde* is removed because *ade* is not in L_3
- C_4={*abcd*}

2.2.6 Methods to Improve Apriori's Efficiency

- Hash-based itemset counting: A *k*-itemset whose corresponding hashing bucket count is below the threshold cannot be frequent
- Transaction reduction: A transaction that does not contain any frequent k-itemset is useless in subsequent scans

www.android.previousquestionpapers.com | www.previousquestionpapers.com | www.ios.previousquestionpapers.com

www.android.universityupdates.in | www.universityupdates.in | www.ios.universityupdates.in

- Partitioning: Any itemset that is potentially frequent in DB must be frequent in at least one of the partitions of DB
- Sampling: mining on a subset of given data, lower support threshold + a method to determine the completeness
- Dynamic itemset counting: add new candidate itemsets only when all of their subsets are estimated to be frequent

2.2.7 Is Apriori Fast Enough? — Performance

- The core of the Apriori algorithm:
  - Use frequent (k – 1)-itemsets to generate candidate frequent k-itemsets
  - Use database scan and pattern matching to collect counts for the candidate itemsets
- The bottleneck of *Apriori*: candidate generation
  - Huge candidate sets:
    - 10^4 frequent 1-itemset will generate 10^7 candidate 2-itemsets
    - To discover a frequent pattern of size 100, e.g., $\{a_1, a_2, \ldots, a_{100}\}$, one needs to generate $2^{100} \approx 10^{30}$ candidates.
  - Multiple scans of database:
    - Needs (n +1) scans, n is the length of the longest pattern

2.3 FP-Tree Growth Algorithm :(Mining Frequent Patterns) Without Candidate Generation

- Compress a large database into a compact, Frequent-Pattern tree (FP-tree) structure
  - highly condensed, but complete for frequent pattern mining
  - avoid costly database scans
- Develop an efficient, FP-tree-based frequent pattern mining method
  - A divide-and-conquer methodology: decompose mining tasks into smaller ones
  - Avoid candidate generation: sub-database test only!

2.3.1 Construct FP-tree from a Transaction DB

www.android.previousquestionpapers.com | www.previousquestionpapers.com | www.ios.previousquestionpapers.com

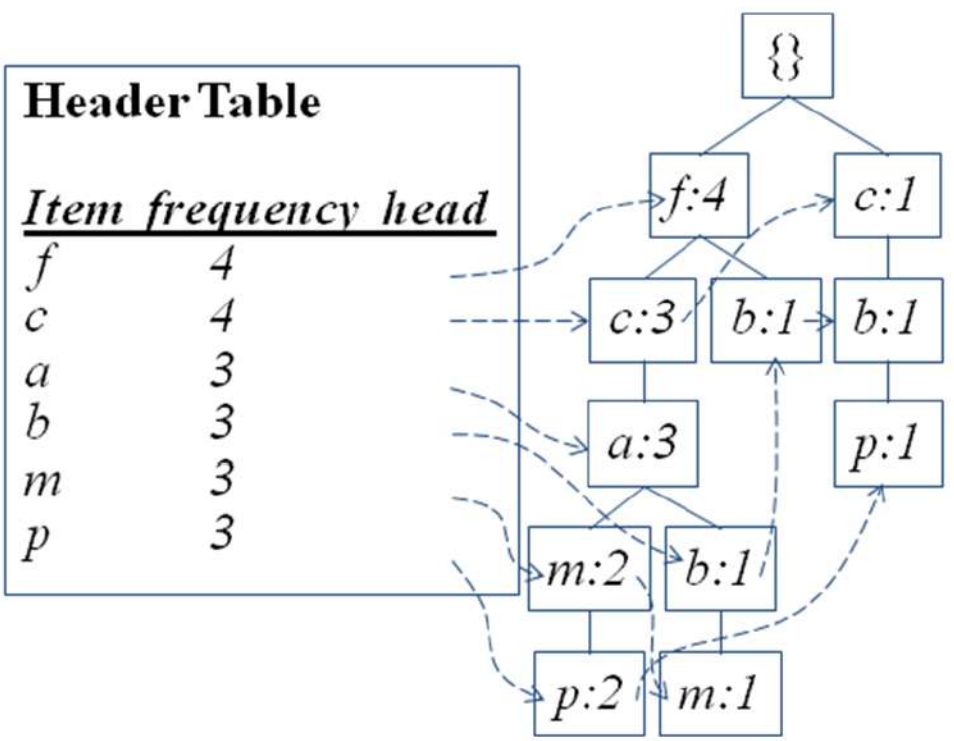
www.android.universityupdates.in | www.universityupdates.in | www.ios.universityupdates.in

TID	Items bought	(ordered) frequent items
100	{f, a, c, d, g, i, m, p}	{f, c, a, m, p}
200	{a, b, c, f, l, m, o}	{f, c, a, b, m}
300	{b, f, h, j, o}	{f, b}
400	{b, c, k, s, p}	{c, b, p}
500	{a, f, c, e, l, p, m, n}	{f, c, a, m, p}

$min_support = 0.5$

Steps:

1. Scan DB once, find frequent 1-itemset (single item pattern)
2. Order frequent items in frequency descending order
3. Scan DB again, construct FP-tree



2.3.2 Benefits of the FP-tree Structure

- Completeness:
  - never breaks a long pattern of any transaction
  - preserves complete information for frequent pattern mining
- Compactness
  - reduce irrelevant information—infrequent items are gone

www.android.previousquestionpapers.com | www.previousquestionpapers.com | www.ios.previousquestionpapers.com

www.android.universityupdates.in | www.universityupdates.in | www.ios.universityupdates.in

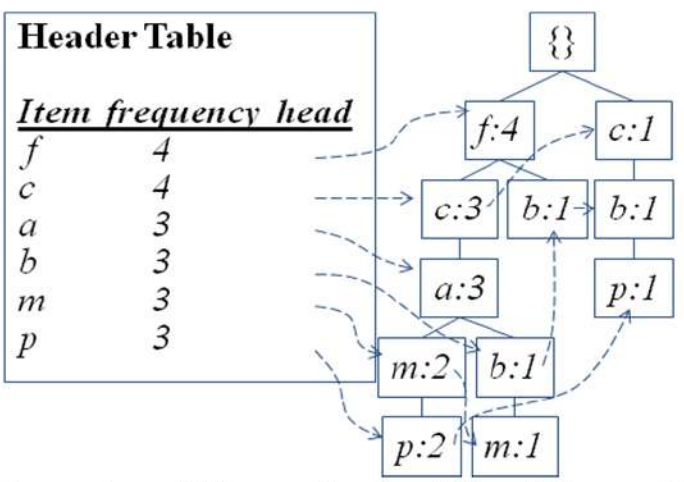
- frequency descending ordering: more frequent items are more likely to be shared
- never be larger than the original database (if not count node-links and counts)
- Example: For Connect-4 DB, compression ratio could be over 100

2.3.3 Mining Frequent Patterns Using FP-tree

- General idea (divide-and-conquer)
  - Recursively grow frequent pattern path using the FP-tree
- Method
  - For each item, construct its conditional pattern-base, and then its conditional FP-tree
  - Repeat the process on each newly created conditional FP-tree
  - Until the resulting FP-tree is empty, or it contains only one path (single path will generate all the combinations of its sub-paths, each of which is a frequent pattern)

Step 1: From FP-tree to Conditional Pattern Base

- Starting at the frequent header table in the FP-tree
- Traverse the FP-tree by following the link of each frequent item
- Accumulate all of transformed prefix paths of that item to form a conditional pattern base



Conditional **pattern bases**

item	*cond. pattern base*
c	*f:3*
a	*fc:3*
b	*fca:1, f:1, c:1*
m	*fca:2, fcab:1*
p	*fcam:2, cb:1*

Properties of FP-tree for Conditional Pattern Base Construction

- Node-link property
  - For any frequent item a_i, all the possible frequent patterns that contain a_i can be obtained by following a_i's node-links, starting from a_i's head in the FP-tree header

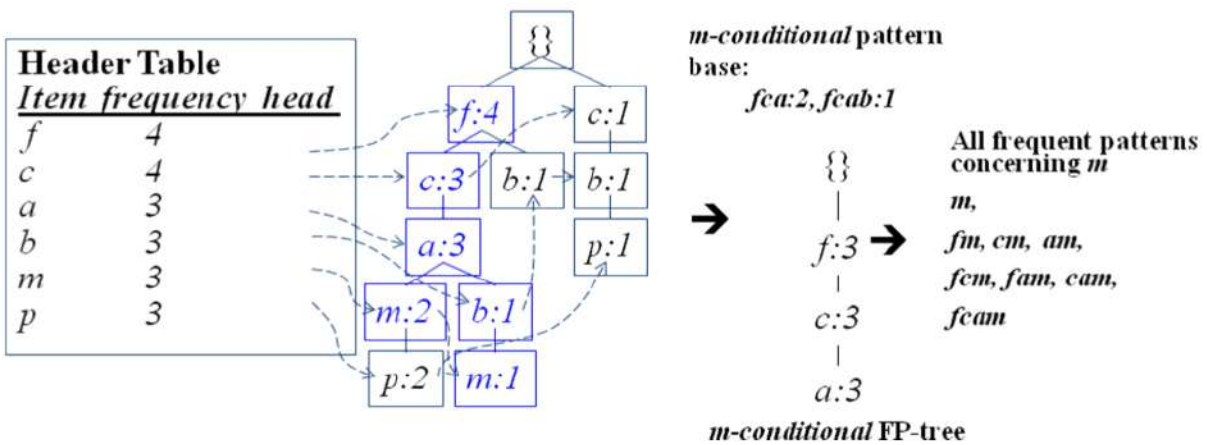
www.android.previousquestionpapers.com | www.previousquestionpapers.com | www.ios.previousquestionpapers.com

www.android.universityupdates.in | www.universityupdates.in | www.ios.universityupdates.in

- Prefix path property
  - To calculate the frequent patterns for a node a_i in a path P, only the prefix sub-path of a_i in P need to be accumulated, and its frequency count should carry the same count as node a_i.

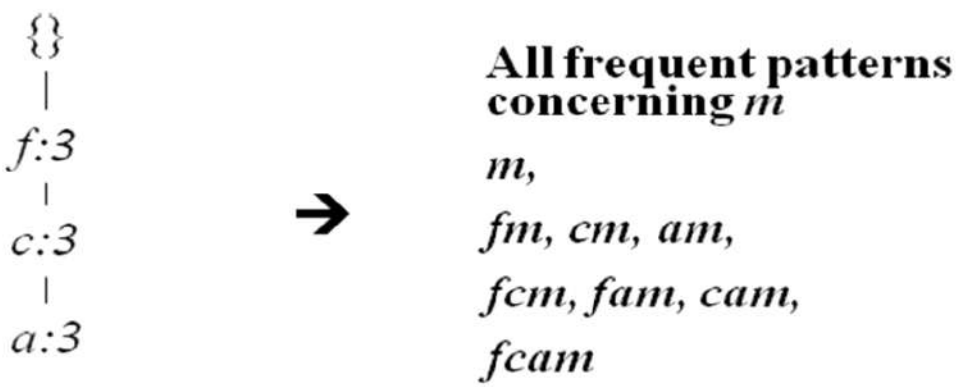
Step 2: Construct Conditional FP-tree

- For each pattern-base
  - Accumulate the count for each item in the base
  - Construct the FP-tree for the frequent items of the pattern base



2.3.4 Single FP-tree Path Generation

- Suppose an FP-tree T has a single path P
- The complete set of frequent pattern of T can be generated by enumeration of all the combinations of the sub-paths of P



m-conditional FP-tree

www.android.previousquestionpapers.com | www.previousquestionpapers.com | www.ios.previousquestionpapers.com

www.android.universityupdates.in | www.universityupdates.in | www.ios.universityupdates.in

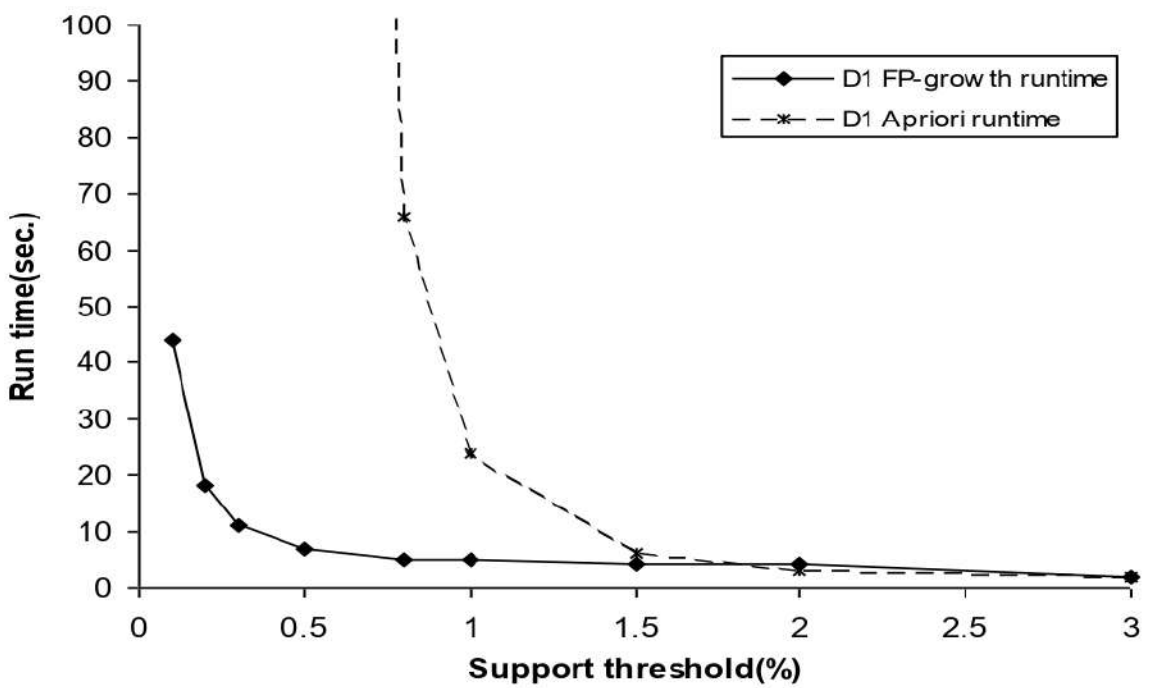
2.3.5 Principles of Frequent Pattern Growth

- Pattern growth property
  - Let α be a frequent itemset in DB, B be α's conditional pattern base, and β be an itemset in B. Then $\alpha \cup \beta$ is a frequent itemset in DB iff β is frequent in B.
- "*abcdef*" is a frequent pattern, if and only if
  - "*abcde*" is a frequent pattern, and
  - "*f*" is frequent in the set of transactions containing "*abcde*"

2.3.6 Why Is Frequent Pattern Growth Fast?

- performance study shows
  - FP-growth is an order of magnitude faster than Apriori, and is also faster than tree-projection
- Reasoning
  - No candidate generation, no candidate test
  - Use compact data structure
  - Eliminate repeated database scan
  - Basic operation is counting and FP-tree building

2.3.7 FP-growth vs. Apriori: Scalability With the Support Threshold



[

www.android.previousquestionpapers.com | www.previousquestionpapers.com | www.ios.previousquestionpapers.com

www.android.universityupdates.in | www.universityupdates.in | www.ios.universityupdates.in

2.4 Compact Representation of Frequent Itemset

- Usually, huge number of frequent itemsets produced from transactional dataset.
- It is useful to identify a **small representative set of itemsets** from which all **other frequent itemsets** can be derived.
- There are two such representations

1. **Maximal Frequent Itemsets:**

   - An itemset is maximal frequent if none of its immediate supersets is frequent.

2. **Closed Frequent Itemsets:**

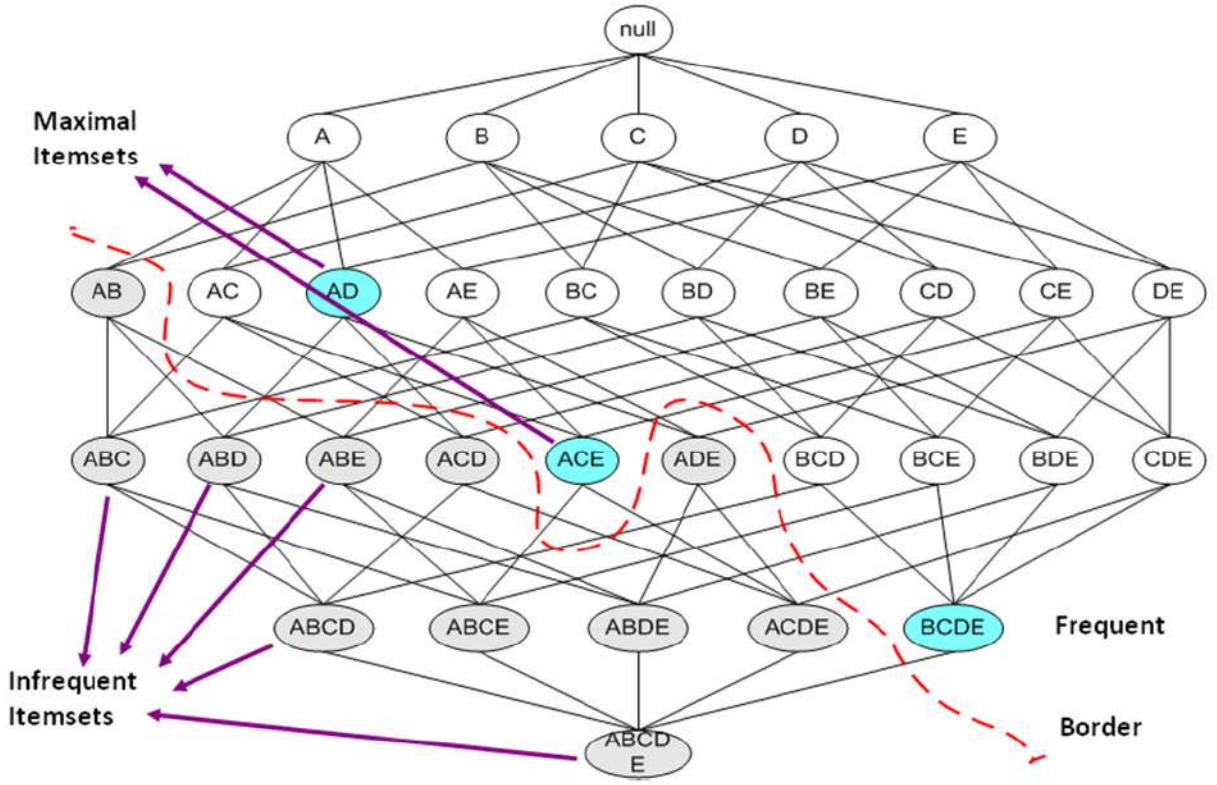
   - An itemset is closed if none of it's immediate supersets has exactly the same support count .
   - Closed itemsets provide a minimal representation of itemsets without losing their support information.

2.5 Maximal Frequent Itemset

An itemset is maximal frequent if none of its immediate supersets is frequent.

www.android.previousquestionpapers.com | www.previousquestionpapers.com | www.ios.previousquestionpapers.com

www.android.universityupdates.in | www.universityupdates.in | www.ios.universityupdates.in



2.6 Closed Frequent Itemsets

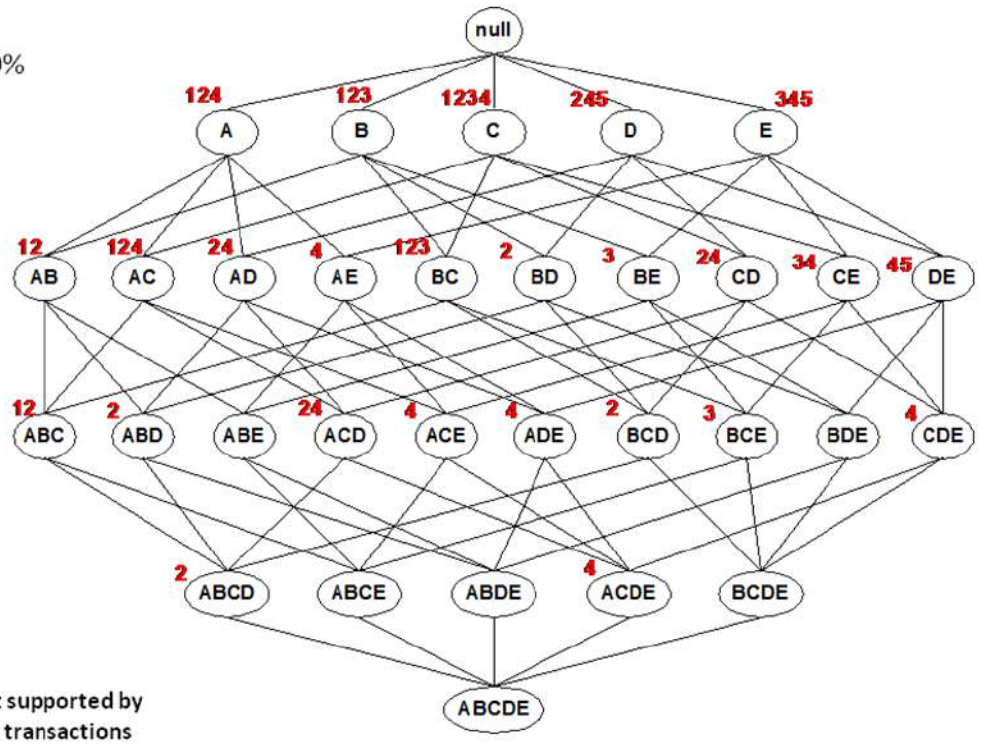
www.android.previousquestionpapers.com | www.previousquestionpapers.com | www.ios.previousquestionpapers.com

www.android.universityupdates.in | www.universityupdates.in | www.ios.universityupdates.in

Min_support%=40%

TID	Items
1	ABC
2	ABCD
3	BCE
4	ACDE
5	DE

C,D,E,AC,BC,CE,DE,ABC,ACD are the **closed Frequent Itemset**



Not supported by any transactions

www.android.previousquestionpapers.com | www.previousquestionpapers.com | www.ios.previousquestionpapers.com

current itemset S_c can be subsumed by another already found closed itemset S_a, then (1) S_c and S_a have the same support, (2) the length of S_c is smaller than that of S_a, and (3) all of the items in S_c are contained in S_a. Based on this property, a **two-level hash index structure** can be built for fast accessing of the pattern-tree: The first level uses the identifier of the last item in S_c as a hash key (since this identifier must be within the branch of S_c), and the second level uses the support of S_c as a hash key (since S_c and S_a have the same support). This will substantially speed up the subset checking process.

The above discussion illustrates methods for efficient mining of closed frequent itemsets. *"Can we extend these methods for efficient mining of maximal frequent itemsets?"* Because maximal frequent itemsets share many similarities with closed frequent itemsets, many of the optimization techniques developed here can be extended to mining maximal frequent itemsets. However, we leave this method as an exercise for interested readers.

5.3 Mining Various Kinds of Association Rules

We have studied efficient methods for mining frequent itemsets and association rules. In this section, we consider additional application requirements by extending our scope to include mining multilevel association rules, multidimensional association rules, and quantitative association rules in transactional and/or relational databases and data warehouses. *Multilevel association rules* involve concepts at different levels of abstraction. *Multidimensional association rules* involve more than one dimension or predicate (e.g., rules relating what a customer *buys* as well as the customer's *age*.) *Quantitative association rules* involve numeric attributes that have an implicit ordering among values (e.g., *age*).

5.3.1 Mining Multilevel Association Rules

For many applications, it is difficult to find strong associations among data items at low or primitive levels of abstraction due to the sparsity of data at those levels. Strong associations discovered at high levels of abstraction may represent commonsense knowledge. Moreover, what may represent common sense to one user may seem novel to another. Therefore, data mining systems should provide capabilities for mining association rules at multiple levels of abstraction, with sufficient flexibility for easy traversal among different abstraction spaces.

Let's examine the following example.

Example 5.7 **Mining multilevel association rules.** Suppose we are given the task-relevant set of transactional data in Table 5.6 for sales in an *AllElectronics* store, showing the items purchased for each transaction. The concept hierarchy for the items is shown in Figure 5.10. A concept hierarchy defines a sequence of mappings from a set of low-level concepts to higher-level, more general concepts. Data can be generalized by replacing low-level concepts within the data by their higher-level concepts, or *ancestors*, from a concept hierarchy.

Table 5.6 Task-relevant data, D.

TID	Items Purchased
T100	IBM-ThinkPad-T40/2373, HP-Photosmart-7660
T200	Microsoft-Office-Professional-2003, Microsoft-Plus!-Digital-Media
T300	Logitech-MX700-Cordless-Mouse, Fellowes-Wrist-Rest
T400	Dell-Dimension-XPS, Canon-PowerShot-S400
T500	IBM-ThinkPad-R40/P4M, Symantec-Norton-Antivirus-2003
...	...

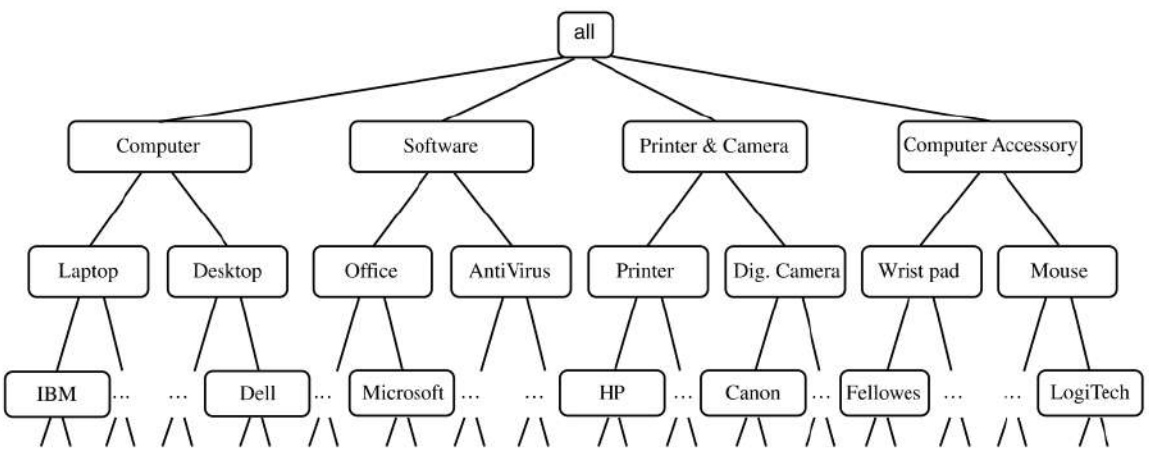


Figure 5.10 A concept hierarchy for *AllElectronics* computer items.

The concept hierarchy of Figure 5.10 has five levels, respectively referred to as levels 0 to 4, starting with level 0 at the root node for **all** (the most general abstraction level). Here, level 1 includes *computer, software, printer&camera,* and *computer accessory*, level 2 includes *laptop computer, desktop computer, office software, antivirus software, ...*, and level 3 includes *IBM desktop computer, ..., Microsoft office software*, and so on. Level 4 is the most specific abstraction level of this hierarchy. It consists of the raw data values. Concept hierarchies for categorical attributes are often implicit within the database schema, in which case they may be automatically generated using methods such as those described in Chapter 2. For our example, the concept hierarchy of Figure 5.10 was generated from data on product specifications. Concept hierarchies for numerical attributes can be generated using discretization techniques, many of which were introduced in Chapter 2. Alternatively, concept hierarchies may be specified by users familiar with the data, such as store managers in the case of our example.

The items in Table 5.6 are at the lowest level of the concept hierarchy of Figure 5.10. It is difficult to find interesting purchase patterns at such raw or primitive-level data. For instance, if "*IBM-ThinkPad-R40/P4M*" or "*Symantec-Norton-Antivirus-2003*" each

occurs in a very small fraction of the transactions, then it can be difficult to find strong associations involving these specific items. Few people may buy these items together, making it unlikely that the itemset will satisfy minimum support. However, we would expect that it is easier to find strong associations between generalized abstractions of these items, such as between *"IBM laptop computer"* and *"antivirus software."* ■

Association rules generated from mining data at multiple levels of abstraction are called **multiple-level** or **multilevel association rules**. Multilevel association rules can be mined efficiently using concept hierarchies under a support-confidence framework. In general, a top-down strategy is employed, where counts are accumulated for the calculation of frequent itemsets at each concept level, starting at the concept level 1 and working downward in the hierarchy toward the more specific concept levels, until no more frequent itemsets can be found. For each level, any algorithm for discovering frequent itemsets may be used, such as Apriori or its variations. A number of variations to this approach are described below, where each variation involves "playing" with the support threshold in a slightly different way. The variations are illustrated in Figures 5.11 and 5.12, where nodes indicate an item or itemset that has been examined, and nodes with thick borders indicate that an examined item or itemset is frequent.

- **Using uniform minimum support for all levels** (referred to as **uniform support**): The same minimum support threshold is used when mining at each level of abstraction. For example, in Figure 5.11, a minimum support threshold of 5% is used throughout (e.g., for mining from *"computer"* down to *"laptop computer"*). Both *"computer"* and *"laptop computer"* are found to be frequent, while *"desktop computer"* is not.

  When a uniform minimum support threshold is used, the search procedure is simplified. The method is also simple in that users are required to specify only one minimum support threshold. An Apriori-like optimization technique can be adopted, based on the knowledge that an ancestor is a superset of its descendants: The search avoids examining itemsets containing any item whose ancestors do not have minimum support.

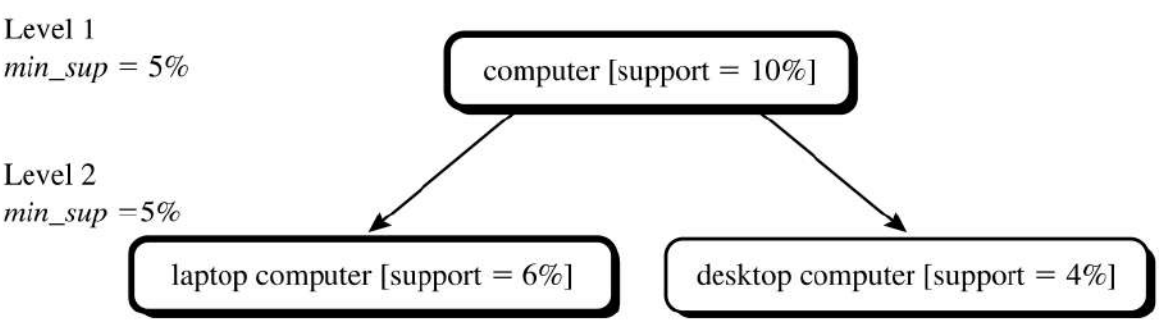


Figure 5.11 Multilevel mining with uniform support.

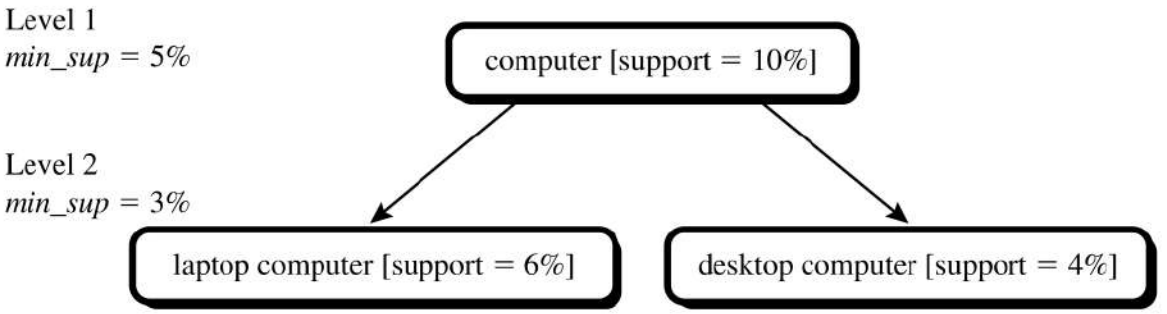


Figure 5.12 Multilevel mining with reduced support.

The uniform support approach, however, has some difficulties. It is unlikely that items at lower levels of abstraction will occur as frequently as those at higher levels of abstraction. If the minimum support threshold is set too high, it could miss some meaningful associations occurring at low abstraction levels. If the threshold is set too low, it may generate many uninteresting associations occurring at high abstraction levels. This provides the motivation for the following approach.

- **Using reduced minimum support at lower levels** (referred to as **reduced support**): Each level of abstraction has its own minimum support threshold. The deeper the level of abstraction, the smaller the corresponding threshold is. For example, in Figure 5.12, the minimum support thresholds for levels 1 and 2 are 5% and 3%, respectively. In this way, "*computer,*" "*laptop computer,*" and "*desktop computer*" are all considered frequent.
- **Using item or group-based minimum support (referred to as group-based support)**: Because users or experts often have insight as to which groups are more important than others, it is sometimes more desirable to set up user-specific, item, or group-based minimal support thresholds when mining multilevel rules. For example, a user could set up the minimum support thresholds based on product price, or on items of interest, such as by setting particularly low support thresholds for *laptop computers* and *flash drives* in order to pay particular attention to the association patterns containing items in these categories.

Notice that the Apriori property may not always hold uniformly across all of the items when mining under reduced support and group-based support. However, efficient methods can be developed based on the extension of the property. The details are left as an exercise for interested readers.

A serious side effect of mining multilevel association rules is its generation of many redundant rules across multiple levels of abstraction due to the "ancestor" relationships among items. For example, consider the following rules where "*laptop computer*" is an ancestor of "*IBM laptop computer*" based on the concept hierarchy of Figure 5.10, and where X is a variable representing customers who purchased items in *AllElectronics* transactions.

$$buys(X, \text{“}laptop\ computer\text{”}) \Rightarrow buys(X, \text{“}HP\ printer\text{”})$$
$$[support = 8\%, confidence = 70\%] \quad (5.10)$$

$$buys(X, \text{“}IBM\ laptop\ computer\text{”}) \Rightarrow buys(X, \text{“}HP\ printer\text{”})$$
$$[support = 2\%, confidence = 72\%] \quad (5.11)$$

“If Rules (5.10) and (5.11) are both mined, then how useful is the latter rule?” you may wonder. *“Does it really provide any novel information?”* If the latter, less general rule does not provide new information, then it should be removed. Let's look at how this may be determined. A rule $R1$ is an **ancestor** of a rule $R2$, if $R1$ can be obtained by replacing the items in $R2$ by their ancestors in a concept hierarchy. For example, Rule (5.10) is an ancestor of Rule (5.11) because *“laptop computer”* is an ancestor of *“IBM laptop computer.”* Based on this definition, a rule can be considered redundant if its support and confidence are close to their “expected” values, based on an ancestor of the rule. As an illustration, suppose that Rule (5.10) has a 70% confidence and 8% support, and that about one-quarter of all *“laptop computer”* sales are for *“IBM laptop computers.”* We may expect Rule (5.11) to have a confidence of around 70% (since all data samples of *“IBM laptop computer”* are also samples of *“laptop computer”*) and a support of around 2% (i.e., $8\% \times \frac{1}{4}$). If this is indeed the case, then Rule (5.11) is not interesting because it does not offer any additional information and is less general than Rule (5.10).

5.3.2 Mining Multidimensional Association Rules from Relational Databases and Data Warehouses

So far in this chapter, we have studied association rules that imply a single predicate, that is, the predicate *buys*. For instance, in mining our *AllElectronics* database, we may discover the Boolean association rule

$$buys(X, \text{“}digital\ camera\text{”}) \Rightarrow buys(X, \text{“}HP\ printer\text{”}). \quad (5.12)$$

Following the terminology used in multidimensional databases, we refer to each distinct predicate in a rule as a dimension. Hence, we can refer to Rule (5.12) as a **single-dimensional** or **intradimensional association rule** because it contains a single distinct predicate (e.g., *buys*) with multiple occurrences (i.e., the predicate occurs more than once within the rule). As we have seen in the previous sections of this chapter, such rules are commonly mined from transactional data.

Suppose, however, that rather than using a transactional database, sales and related information are stored in a relational database or data warehouse. Such data stores are multidimensional, by definition. For instance, in addition to keeping track of the items purchased in sales transactions, a relational database may record other attributes associated with the items, such as the quantity purchased or the price, or the branch location of the sale. Additional relational information regarding the customers who purchased the items, such as customer age, occupation, credit rating, income, and address, may also be

stored. Considering each database attribute or warehouse dimension as a predicate, we can therefore mine association rules containing *multiple* predicates, such as

$$age(X, \text{“20...29”}) \wedge occupation(X, \text{“}student\text{”}) \Rightarrow buys(X, \text{“}laptop\text{”}). \quad (5.13)$$

Association rules that involve two or more dimensions or predicates can be referred to as **multidimensional association rules**. Rule (5.13) contains three predicates (*age, occupation*, and *buys*), each of which occurs *only once* in the rule. Hence, we say that it has **no repeated predicates**. Multidimensional association rules with no repeated predicates are called **interdimensional association rules**. We can also mine multidimensional association rules with repeated predicates, which contain multiple occurrences of some predicates. These rules are called **hybrid-dimensional association rules**. An example of such a rule is the following, where the predicate *buys* is repeated:

$$age(X, \text{“20...29”}) \wedge buys(X, \text{“}laptop\text{”}) \Rightarrow buys(X, \text{“}HP\ printer\text{”}) \quad (5.14)$$

Note that database attributes can be categorical or quantitative. **Categorical** attributes have a finite number of possible values, with no ordering among the values (e.g., *occupation, brand, color*). Categorical attributes are also called **nominal** attributes, because their values are “names of things.” **Quantitative** attributes are numeric and have an implicit ordering among values (e.g., *age, income, price*). Techniques for mining multidimensional association rules can be categorized into two basic approaches regarding the treatment of quantitative attributes.

In the first approach, *quantitative attributes are discretized using predefined concept hierarchies*. This discretization occurs before mining. For instance, a concept hierarchy for *income* may be used to replace the original numeric values of this attribute by interval labels, such as “0...20K”, “21K ...30K”, “31K ...40K”, and so on. Here, discretization is *static* and predetermined. Chapter 2 on data preprocessing gave several techniques for discretizing numeric attributes. The discretized numeric attributes, with their interval labels, can then be treated as categorical attributes (where each interval is considered a category). We refer to this as **mining multidimensional association rules using static discretization of quantitative attributes**.

In the second approach, *quantitative attributes are discretized or clustered into “bins” based on the distribution of the data*. These bins may be further combined during the mining process. The discretization process is *dynamic* and established so as to satisfy some mining criteria, such as maximizing the confidence of the rules mined. Because this strategy treats the numeric attribute values as quantities rather than as predefined ranges or categories, association rules mined from this approach are also referred to as **(dynamic) quantitative association rules**.

Let’s study each of these approaches for mining multidimensional association rules. For simplicity, we confine our discussion to interdimensional association rules. Note that rather than searching for frequent itemsets (as is done for single-dimensional association rule mining), in multidimensional association rule mining we search for frequent *predicate sets*. A k-**predicate set** is a set containing k conjunctive predicates. For instance, the set of predicates $\{age, occupation, buys\}$ from Rule (5.13) is a 3-predicate set. Similar

to the notation used for itemsets, we use the notation L_k to refer to the set of frequent k-predicate sets.

Mining Multidimensional Association Rules Using Static Discretization of Quantitative Attributes

Quantitative attributes, in this case, are discretized before mining using predefined concept hierarchies or data discretization techniques, where numeric values are replaced by interval labels. Categorical attributes may also be generalized to higher conceptual levels if desired. If the resulting task-relevant data are stored in a relational table, then any of the frequent itemset mining algorithms we have discussed can be modified easily so as to find all frequent predicate sets rather than frequent itemsets. In particular, instead of searching on only one attribute like *buys*, we need to search through all of the relevant attributes, treating each attribute-value pair as an itemset.

Alternatively, the transformed multidimensional data may be used to construct a *data cube*. Data cubes are well suited for the mining of multidimensional association rules: They store aggregates (such as counts), in multidimensional space, which is essential for computing the support and confidence of multidimensional association rules. An overview of data cube technology was presented in Chapter 3. Detailed algorithms for data cube computation were given in Chapter 4. Figure 5.13 shows the lattice of cuboids defining a data cube for the dimensions *age*, *income*, and *buys*. The cells of an n-dimensional cuboid can be used to store the support counts of the corresponding

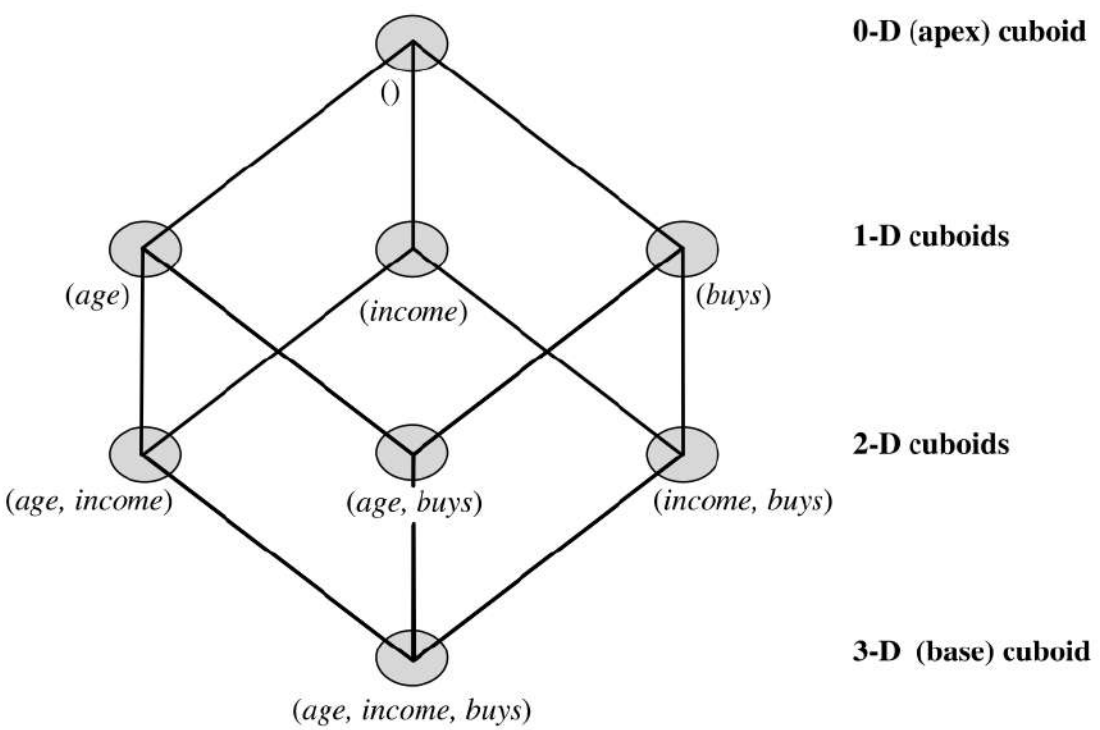


Figure 5.13 Lattice of cuboids, making up a 3-D data cube. Each cuboid represents a different group-by. The base cuboid contains the three predicates *age*, *income*, and *buys*.

n-predicate sets. The base cuboid aggregates the task-relevant data by *age*, *income*, and *buys*; the 2-D cuboid, (*age*, *income*), aggregates by *age* and *income*, and so on; the 0-D (apex) cuboid contains the total number of transactions in the task-relevant data.

Due to the ever-increasing use of data warehouse and OLAP technology, it is possible that a data cube containing the dimensions that are of interest to the user may already exist, fully materialized. If this is the case, we can simply fetch the corresponding aggregate values and return the rules needed using a rule generation algorithm (Section 5.2.2). Notice that even in this case, the Apriori property can still be used to prune the search space. If a given k-predicate set has support *sup*, which does not satisfy minimum support, then further exploration of this set should be terminated. This is because any more specialized version of the k-itemset will have support no greater that *sup* and, therefore, will not satisfy minimum support either. In cases where no relevant data cube exists for the mining task, we must create one on the fly. This becomes an iceberg cube computation problem, where the minimum support threshold is taken as the iceberg condition (Chapter 4).

Mining Quantitative Association Rules

Quantitative association rules are multidimensional association rules in which the numeric attributes are *dynamically* discretized during the mining process so as to satisfy some mining criteria, such as maximizing the confidence or compactness of the rules mined. In this section, we focus specifically on how to mine quantitative association rules having two quantitative attributes on the left-hand side of the rule and one categorical attribute on the right-hand side of the rule. That is,

$$A_{quan1} \wedge A_{quan2} \Rightarrow A_{cat}$$

where A_{quan1} and A_{quan2} are tests on quantitative attribute intervals (where the intervals are dynamically determined), and A_{cat} tests a categorical attribute from the task-relevant data. Such rules have been referred to as **two-dimensional quantitative association rules**, because they contain two quantitative dimensions. For instance, suppose you are curious about the association relationship between pairs of quantitative attributes, like customer age and income, and the type of television (such as *high-definition TV*, i.e., *HDTV*) that customers like to buy. An example of such a 2-D quantitative association rule is

$$age(X, \text{“30...39”}) \wedge income(X, \text{“42}K\text{...48}K\text{”}) \Rightarrow buys(X, \text{“}HDTV\text{”}) \quad (5.15)$$

“How can we find such rules?” Let’s look at an approach used in a system called **ARCS** (Association Rule Clustering System), which borrows ideas from image processing. Essentially, this approach maps pairs of quantitative attributes onto a 2-D grid for tuples satisfying a given categorical attribute condition. The grid is then searched for clusters of points from which the association rules are generated. The following steps are involved in ARCS:

Binning: Quantitative attributes can have a very wide range of values defining their domain. Just think about how big a 2-D grid would be if we plotted *age* and *income* as

axes, where each possible value of *age* was assigned a unique position on one axis, and similarly, each possible value of *income* was assigned a unique position on the other axis! To keep grids down to a manageable size, we instead partition the ranges of quantitative attributes into intervals. These intervals are dynamic in that they may later be further combined during the mining process. The partitioning process is referred to as **binning**, that is, where the intervals are considered "bins." Three common binning strategies area as follows:

- **Equal-width binning**, where the interval size of each bin is the same
- **Equal-frequency binning**, where each bin has approximately the same number of tuples assigned to it,
- **Clustering-based binning**, where clustering is performed on the quantitative attribute to group *neighboring points* (judged based on various distance measures) into the same bin

ARCS uses equal-width binning, where the bin size for each quantitative attribute is input by the user. A 2-D array for each possible bin combination involving both quantitative attributes is created. Each array cell holds the corresponding count distribution for each possible class of the categorical attribute of the rule right-hand side. By creating this data structure, the task-relevant data need only be scanned once. The same 2-D array can be used to generate rules for any value of the categorical attribute, based on the same two quantitative attributes. Binning is also discussed in Chapter 2.

Finding frequent predicate sets: Once the 2-D array containing the count distribution for each category is set up, it can be scanned to find the frequent predicate sets (those satisfying minimum support) that also satisfy minimum confidence. Strong association rules can then be generated from these predicate sets, using a rule generation algorithm like that described in Section 5.2.2.

Clustering the association rules: The strong association rules obtained in the previous step are then mapped to a 2-D grid. Figure 5.14 shows a 2-D grid for 2-D quantitative association rules predicting the condition *buys*(*X*, "*HDTV*") on the rule right-hand side, given the quantitative attributes *age* and *income*. The four Xs correspond to the rules

$$age(X, 34) \wedge income(X, \text{"}31K...40K\text{"}) \Rightarrow buys(X, \text{"}HDTV\text{"}) \quad (5.16)$$

$$age(X, 35) \wedge income(X, \text{"}31K...40K\text{"}) \Rightarrow buys(X, \text{"}HDTV\text{"}) \quad (5.17)$$

$$age(X, 34) \wedge income(X, \text{"}41K...50K\text{"}) \Rightarrow buys(X, \text{"}HDTV\text{"}) \quad (5.18)$$

$$age(X, 35) \wedge income(X, \text{"}41K...50K\text{"}) \Rightarrow buys(X, \text{"}HDTV\text{"}). \quad (5.19)$$

"*Can we find a simpler rule to replace the above four rules?*" Notice that these rules are quite "close" to one another, forming a rule cluster on the grid. Indeed, the four rules can be combined or "clustered" together to form the following simpler rule, which subsumes and replaces the above four rules:

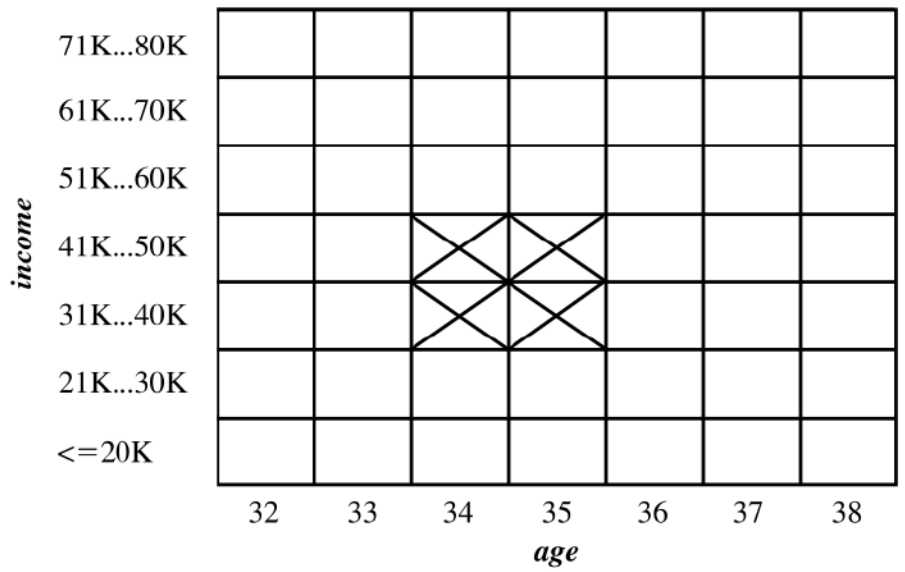


Figure 5.14 A 2-D grid for tuples representing customers who purchase high-definition TVs.

$$age(X, \text{“34...35”}) \wedge income(X, \text{“31}K\text{...50}K\text{”}) \Rightarrow buys(X, \text{“}HDTV\text{”}) \quad (5.20)$$

ARCS employs a clustering algorithm for this purpose. The algorithm scans the grid, searching for rectangular clusters of rules. In this way, bins of the quantitative attributes occurring within a rule cluster may be further combined, and hence further dynamic discretization of the quantitative attributes occurs.

The grid-based technique described here assumes that the initial association rules can be clustered into rectangular regions. Before performing the clustering, smoothing techniques can be used to help remove noise and outliers from the data. Rectangular clusters may oversimplify the data. Alternative approaches have been proposed, based on other shapes of regions that tend to better fit the data, yet require greater computation effort.

A non-grid-based technique has been proposed to find quantitative association rules that are more general, where any number of quantitative and categorical attributes can appear on either side of the rules. In this technique, quantitative attributes are dynamically partitioned using equal-frequency binning, and the partitions are combined based on a measure of *partial completeness,* which quantifies the information lost due to partitioning. For references on these alternatives to ARCS, see the bibliographic notes.

5.4 From Association Mining to Correlation Analysis

Most association rule mining algorithms employ a support-confidence framework. Often, many interesting rules can be found using low support thresholds. Although minimum support and confidence thresholds *help* weed out or exclude the exploration

of a good number of uninteresting rules, many rules so generated are still not interesting to the users. Unfortunately, this is especially true *when mining at low support thresholds or mining for long patterns.* This has been one of the major bottlenecks for successful application of association rule mining.

In this section, we first look at how even strong association rules can be uninteresting and misleading. We then discuss how the support-confidence framework can be supplemented with additional interestingness measures based on statistical significance and correlation analysis.

5.4.1 Strong Rules Are Not Necessarily Interesting: An Example

Whether or not a rule is interesting can be assessed either subjectively or objectively. Ultimately, only the user can judge if a given rule is interesting, and this judgment, being subjective, may differ from one user to another. However, objective interestingness measures, based on the statistics "behind" the data, can be used as one step toward the goal of weeding out uninteresting rules from presentation to the user.

"How can we tell which strong association rules are really interesting?" Let's examine the following example.

Example 5.8 **A misleading "strong" association rule.** Suppose we are interested in analyzing transactions at *AllElectronics* with respect to the purchase of computer games and videos. Let *game* refer to the transactions containing computer games, and *video* refer to those containing videos. Of the 10,000 transactions analyzed, the data show that 6,000 of the customer transactions included computer games, while 7,500 included videos, and 4,000 included both computer games and videos. Suppose that a data mining program for discovering association rules is run on the data, using a minimum support of, say, 30% and a minimum confidence of 60%. The following association rule is discovered:

$$buys(X, \text{"}computer\ games\text{"}) \Rightarrow buys(X, \text{"}videos\text{"}) \quad [support = 40\%, confidence = 66\%] \tag{5.21}$$

Rule (5.21) is a strong association rule and would therefore be reported, since its support value of $\frac{4{,}000}{10{,}000} = 40\%$ and confidence value of $\frac{4{,}000}{6{,}000} = 66\%$ satisfy the minimum support and minimum confidence thresholds, respectively. However, Rule (5.21) is misleading because the probability of purchasing videos is 75%, which is even larger than 66%. In fact, computer games and videos are negatively associated because the purchase of one of these items actually decreases the likelihood of purchasing the other. Without fully understanding this phenomenon, we could easily make unwise business decisions based on Rule (5.21). ■

The above example also illustrates that the confidence of a rule $A \Rightarrow B$ can be deceiving in that it is only an *estimate* of the conditional probability of itemset B given itemset A. It does not measure the real strength (or lack of strength) of the correlation and implication between A and B. Hence, alternatives to the support-confidence framework can be useful in mining interesting data relationships.

5.4.2 From Association Analysis to Correlation Analysis

As we have seen above, the support and confidence measures are insufficient at filtering out uninteresting association rules. To tackle this weakness, a correlation measure can be used to augment the support-confidence framework for association rules. This leads to *correlation rules* of the form

$$A \Rightarrow B \; [support, confidence. correlation]. \tag{5.22}$$

That is, a correlation rule is measured not only by its support and confidence but also by the correlation between itemsets A and B. There are many different correlation measures from which to choose. In this section, we study various correlation measures to determine which would be good for mining large data sets.

Lift is a simple correlation measure that is given as follows. The occurrence of itemset A is **independent** of the occurrence of itemset B if $P(A \cup B) = P(A)P(B)$; otherwise, itemsets A and B are **dependent** and **correlated** as events. This definition can easily be extended to more than two itemsets. The **lift** between the occurrence of A and B can be measured by computing

$$lift(A, B) = \frac{P(A \cup B)}{P(A)P(B)}. \tag{5.23}$$

If the resulting value of Equation (5.23) is less than 1, then the occurrence of A is *negatively correlated with* the occurrence of B. If the resulting value is greater than 1, then A and B are *positively correlated*, meaning that the occurrence of one implies the occurrence of the other. If the resulting value is equal to 1, then A and B are *independent* and there is no correlation between them.

Equation (5.23) is equivalent to $P(B|A)/P(B)$, or $conf(A \Rightarrow B)/sup(B)$, which is also referred as the *lift* of the association (or correlation) rule $A \Rightarrow B$. In other words, it assesses the degree to which the occurrence of one "lifts" the occurrence of the other. For example, if A corresponds to the sale of computer games and B corresponds to the sale of videos, then given the current market conditions, the sale of games is said to increase or "lift" the likelihood of the sale of videos by a factor of the value returned by Equation (5.23).

Let's go back to the computer game and video data of Example 5.8.

Example 5.9 **Correlation analysis using lift.** To help filter out misleading "strong" associations of the form $A \Rightarrow B$ from the data of Example 5.8, we need to study how the two itemsets, A and B, are correlated. Let $\overline{game}$ refer to the transactions of Example 5.8 that do not contain computer games, and $\overline{video}$ refer to those that do not contain videos. The transactions can be summarized in a *contingency table*, as shown in Table 5.7. From the table, we can see that the probability of purchasing a computer game is $P(\{game\}) = 0.60$, the probability of purchasing a video is $P(\{video\}) = 0.75$, and the probability of purchasing both is $P(\{game, video\}) = 0.40$. By Equation (5.23), the lift of Rule (5.21) is $P(\{game, video\})/(P(\{game\}) \times P(\{video\})) = 0.40/(0.60 \times 0.75) = 0.89$. Because this value is less than 1, there is a negative correlation between the occurrence of $\{game\}$ and $\{video\}$. The numerator is the likelihood of a customer purchasing both, while the

Table 5.7 A 2 × 2 contingency table summarizing the transactions with respect to game and video purchases.

	game	$\overline{game}$	Σ_{row}
video	4,000	3,500	7,500
$\overline{video}$	2,000	500	2,500
Σ_{col}	6,000	4,000	10,000

Table 5.8 The above contingency table, now shown with the expected values.

	game	$\overline{game}$	Σ_{row}
video	4,000 (4,500)	3,500 (3,000)	7,500
$\overline{video}$	2,000 (1,500)	500 (1,000)	2,500
Σ_{col}	6,000	4,000	10,000

denominator is what the likelihood would have been if the two purchases were completely independent. Such a negative correlation cannot be identified by a support-confidence framework. ■

The second correlation measure that we study is the χ^2 measure, which was introduced in Chapter 2 (Equation 2.9). To compute the χ^2 value, we take the squared difference between the observed and expected value for a slot (A and B pair) in the contingency table, divided by the expected value. This amount is summed for all slots of the contingency table. Let's perform a χ^2 analysis of the above example.

Example 5.10 **Correlation analysis using χ^2.** To compute the correlation using χ^2 analysis, we need the observed value and expected value (displayed in parenthesis) for each slot of the contingency table, as shown in Table 5.8. From the table, we can compute the χ^2 value as follows:

$$\chi^2 = \Sigma \frac{(observed - expected)^2}{expected} = \frac{(4{,}000 - 4{,}500)^2}{4{,}500} + \frac{(3{,}500 - 3{,}000)^2}{3{,}000} + \frac{(2{,}000 - 1{,}500)^2}{1{,}500} + \frac{(500 - 1{,}000)^2}{1{,}000} = 555.6.$$

Because the χ^2 value is greater than one, and the observed value of the slot (*game*, *video*) = 4,000, which is less than the expected value 4,500, *buying game* and *buying video* are *negatively correlated*. This is consistent with the conclusion derived from the analysis of the *lift* measure in Example 5.9. ■

Let's examine two other correlation measures, *all_confidence* and *cosine*, as defined below.

Given an itemset $X = \{i_1, i_2, \ldots, i_k\}$, the **all_confidence** of X is defined as

$$all_conf(X) = \frac{sup(X)}{max_item_sup(X)} = \frac{sup(X)}{max\{sup(i_j)|\forall i_j \in X\}}, \tag{5.24}$$

where $max\{sup(i_j)|\forall i_j \in X\}$ is the maximum (single) item support of all the items in X, and hence is called the **max_item_sup** of the itemset X. The *all_confidence* of X is the minimal confidence among the set of rules $i_j \rightarrow X - i_j$, where $i_j \in X$.

Given two itemsets A and B, the **cosine** measure of A and B is defined as

$$cosine(A, B) = \frac{P(A \cup B)}{\sqrt{P(A) \times P(B)}} = \frac{sup(A \cup B)}{\sqrt{sup(A) \times sup(B)}}. \tag{5.25}$$

The *cosine* measure can be viewed as a harmonized *lift* measure: the two formulae are similar except that for cosine, the *square root* is taken on the product of the probabilities of A and B. This is an important difference, however, because by taking the square root, the cosine value is only influenced by the supports of A, B, and $A \cup B$, and not by the total number of transactions.

"Are these two measures better than lift and χ^2 in assessing the correlation relationship?" To answer this question, we first look at some other typical data sets before returning to our running example.

Example 5.11 **Comparison of four correlation measures on typical data sets.** The correlation relationships between the purchases of two items, *milk* and *coffee*, can be examined by summarizing their purchase history in the form of Table 5.9, a 2×2 contingency table, where an entry such as mc represents the number of transactions containing both milk and coffee. For the derivation of *all_confidence*, we let itemset $X = \{m, c\}$ so that $sup(X) = mc$ in Equation (5.24).

Table 5.10 shows a set of transactional data sets with their corresponding contingency tables and values for each of the four correlation measures. From the table, we see that m and c are positively correlated in A_1 through A_4, independent in B_1, and negatively correlated in C_1 through C_3. All four measures are good indicators for the independent case, B_1. *Lift* and χ^2 are poor indicators of the other relationships, whereas *all_confidence* and *cosine* are good indicators. Another interesting fact is that between *all_confidence* and *cosine*, *cosine* is the better indicator when $\overline{m}c$ and $m\overline{c}$ are not balanced. This is because *cosine* considers the supports of both A and B, whereas *all_confidence* considers only the

Table 5.9 A 2×2 contingency table for two items.

	milk	$\overline{milk}$	Σ_{row}
coffee	mc	$\overline{m}c$	c
$\overline{coffee}$	$m\overline{c}$	$\overline{mc}$	$\overline{c}$
Σ_{col}	m	$\overline{m}$	Σ

Table 5.10 Comparison of four correlation measures using contingency tables for different data sets.

Data Set	mc	$\overline{m}c$	$m\overline{c}$	$\overline{mc}$	*all_conf.*	*cosine*	*lift*	χ^2
A_1	1,000	100	100	100,000	0.91	0.91	83.64	83,452.6
A_2	1,000	100	100	10,000	0.91	0.91	9.26	9,055.7
A_3	1,000	100	100	1,000	0.91	0.91	1.82	1,472.7
A_4	1,000	100	100	0	0.91	0.91	0.99	9.9
B_1	1,000	1,000	1,000	1,000	0.50	0.50	1.00	0.0
C_1	100	1,000	1,000	100,000	0.09	0.09	8.44	670.0
C_2	1,000	100	10,000	100,000	0.09	0.29	9.18	8,172.8
C_3	1	1	100	10,000	0.01	0.07	50.0	48.5

maximal support. Such a difference can be seen by comparing C_1 and C_2. C_1 should be more negatively correlated for m and c than C_2 because mc is the smallest among the three counts, mc, $\overline{m}c$, and $m\overline{c}$, in C_1. However, this can only be seen by checking the *cosine* measure because the *all_confidence* values are identical in C_1 and C_2.

"Why are lift and χ^2 so poor at distinguishing correlation relationships in the above transactional data sets?" To answer this, we have to consider the *null-transactions*. A **null-transaction** is a transaction that does not contain any of the itemsets being examined. In our example, $\overline{mc}$ represents the number of null-transactions. Lift and χ^2 have difficulty distinguishing correlation relationships because they are both strongly influenced by $\overline{mc}$. Typically, the number of null-transactions can outweigh the number of individual purchases, because many people may buy neither milk nor coffee. On the other hand, *all_confidence* and *cosine* values are good indicators of correlation because their definitions remove the influence of $\overline{mc}$ (i.e., they are not influenced by the number of null-transactions). ■

A measure is **null-invariant** if its value is free from the influence of null-transactions. Null-invariance is an important property for measuring correlations in large transaction databases. Among the four above measures, *all_confidence* and *cosine* are null-invariant measures.

"Are all_confidence and cosine the best at assessing correlation in all cases?" Let's examine the game-and-video examples again.

Example 5.12 Comparison of four correlation meaures on game-and-video data. We revisit Examples 5.4.1 to 5.4.2. Let D_1 be the original game (g) and video (v) data set from Table 5.7. We add two more data sets, D_0 and D_2, where D_0 has zero null-transactions, and D_2 has 10,000 null-transactions (instead of only 500 as in D_1). The values of all four correlation measures are shown in Table 5.11.

In Table 5.11, gv, $\overline{g}v$, and $g\overline{v}$ remain the same in D_0, D_1, and D_2. However, lift and χ^2 change from rather negative to rather positive correlations, whereas *all_confidence* and *cosine* have the nice null-invariant property, and their values remain the same in

Table 5.11 Comparison of the four correlation measures for game-and-video data sets.

Data Set	gv	$\overline{g}v$	$g\overline{v}$	$\overline{gv}$	all_conf.	cosine	lift	χ^2
D_0	4,000	3,500	2,000	0	0.53	0.60	0.84	1,477.8
D_1	4,000	3,500	2,000	500	0.53	0.60	0.89	555.6
D_2	4,000	3,500	2,000	10,000	0.53	0.60	1.73	2,913.0

all cases. Unfortunately, we cannot precisely assert that a set of items are positively or negatively correlated when the value of *all_confidence* or *cosine* is around 0.5. Strictly based on whether the value is greater than 0.5, we will claim that g and v are positively correlated in D_1; however, it has been shown that they are negatively correlated by the lift and χ^2 analysis. Therefore, a good strategy is to perform the *all_confidence* or *cosine* analysis first, and when the result shows that they are *weakly* postively/negatively correlated, other analyses can be performed to assist in obtaining a more complete picture. ■

Besides null-invariance, another nice feature of the *all_confidence* measure is that it has the Apriori-like *downward closure* property. That is, if a pattern is *all-confident* (i.e., passing a minimal *all_confidence* threshold), so is every one of its subpatterns. In other words, if a pattern is not all-confident, further growth (or specialization) of this pattern will never satisfy the minimal *all_confidence* threshold. This is obvious since according to Equation (5.24), adding any item into an itemset X will never increase $sup(X)$, never decrease $max_item_sup(X)$, and thus never increase $all_conf(X)$. This property makes Apriori-like pruning possible: we can prune any patterns that cannot satisfy the minimal *all_confidence* threshold during the growth of all-confident patterns in mining.

In summary, the use of only support and confidence measures to mine associations results in the generation of a large number of rules, most of which are uninteresting to the user. Instead, we can augment the support-confidence framework with a correlation measure, resulting in the mining of *correlation rules*. The added measure substantially reduces the number of rules generated, and leads to the discovery of more meaningful rules. However, there seems to be no single correlation measure that works well for all cases. Besides those introduced in this section, many other interestingness measures have been studied in the literature. Unfortunately, most such measures do not have the null-invariance property. Because large data sets typically have many null-transactions, it is important to consider the null-invariance property when selecting appropriate interestingness measures in the correlation analysis. Our analysis shows that both *all_confidence* and *cosine* are good correlation measures for large applications, although it is wise to augment them with additional tests, such as *lift*, when the test result is not conclusive.

5.5 Constraint-Based Association Mining

A data mining process may uncover thousands of rules from a given set of data, most of which end up being unrelated or uninteresting to the users. Often, users have a good sense

of which "direction" of mining may lead to interesting patterns and the "form" of the patterns or rules they would like to find. Thus, a good heuristic is to have the users specify such intuition or expectations as *constraints* to confine the search space. This strategy is known as **constraint-based mining**. The constraints can include the following:

- **Knowledge type constraints:** These specify the type of knowledge to be mined, such as association or correlation.
- **Data constraints:** These specify the set of task-relevant data.
- **Dimension/level constraints:** These specify the desired dimensions (or attributes) of the data, or levels of the concept hierarchies, to be used in mining.
- **Interestingness constraints:** These specify thresholds on statistical measures of rule interestingness, such as support, confidence, and correlation.
- **Rule constraints:** These specify the form of rules to be mined. Such constraints may be expressed as metarules (rule templates), as the maximum or minimum number of predicates that can occur in the rule antecedent or consequent, or as relationships among attributes, attribute values, and/or aggregates.

The above constraints can be specified using a high-level declarative data mining query language and user interface.

The first four of the above types of constraints have already been addressed in earlier parts of this book and chapter. In this section, we discuss the use of *rule constraints* to focus the mining task. This form of constraint-based mining allows users to describe the rules that they would like to uncover, thereby making the data mining process more *effective*. In addition, a sophisticated mining query optimizer can be used to exploit the constraints specified by the user, thereby making the mining process more *efficient*. Constraint-based mining encourages interactive exploratory mining and analysis. In Section 5.5.1, you will study metarule-guided mining, where syntactic rule constraints are specified in the form of rule templates. Section 5.5.2 discusses the use of additional rule constraints, specifying set/subset relationships, constant initiation of variables, and aggregate functions. For ease of discussion, we assume that the user is searching for association rules. The procedures presented can easily be extended to the mining of correlation rules by adding a correlation measure of interestingness to the support-confidence framework, as described in the previous section.

5.5.1 Metarule-Guided Mining of Association Rules

"How are metarules useful?" Metarules allow users to specify the syntactic form of rules that they are interested in mining. The rule forms can be used as constraints to help improve the efficiency of the mining process. Metarules may be based on the analyst's experience, expectations, or intuition regarding the data or may be automatically generated based on the database schema.

Example 5.13 **Metarule-guided mining.** Suppose that as a market analyst for *AllElectronics*, you have access to the data describing customers (such as customer age, address, and credit rating) as well as the list of customer transactions. You are interested in finding associations between customer traits and the items that customers buy. However, rather than finding *all* of the association rules reflecting these relationships, you are particularly interested only in determining which pairs of customer traits promote the sale of office software. A metarule can be used to specify this information describing the form of rules you are interested in finding. An example of such a metarule is

$$P_1(X, Y) \wedge P_2(X, W) \Rightarrow buys(X, \text{“}office\ software\text{”}), \tag{5.26}$$

where P_1 and P_2 are **predicate variables** that are instantiated to attributes from the given database during the mining process, X is a variable representing a customer, and Y and W take on values of the attributes assigned to P_1 and P_2, respectively. Typically, a user will specify a list of attributes to be considered for instantiation with P_1 and P_2. Otherwise, a default set may be used.

In general, a metarule forms a hypothesis regarding the relationships that the user is interested in probing or confirming. The data mining system can then search for rules that match the given metarule. For instance, Rule (5.27) matches or **complies with** Metarule (5.26).

$$age(X, \text{“}30...39\text{”}) \wedge income(X, \text{“}41K...60K\text{”}) \Rightarrow buys(X, \text{“}office\ software\text{”}) \tag{5.27}$$

■

“How can metarules be used to guide the mining process?” Let's examine this problem closely. Suppose that we wish to mine interdimensional association rules, such as in the example above. A metarule is a rule template of the form

$$P_1 \wedge P_2 \wedge \cdots \wedge P_l \Rightarrow Q_1 \wedge Q_2 \wedge \cdots \wedge Q_r, \tag{5.28}$$

where P_i $(i = 1, \ldots, l)$ and Q_j $(j = 1, \ldots, r)$ are either instantiated predicates or predicate variables. Let the number of predicates in the metarule be $p = l + r$. In order to find interdimensional association rules satisfying the template,

- We need to find all frequent p-predicate sets, L_p.
- We must also have the support or count of the l-predicate subsets of L_p in order to compute the confidence of rules derived from L_p.

This is a typical case of mining multidimensional association rules, which was discussed in Section 5.3.2. By extending such methods using techniques described in the following section, we can derive efficient methods for metarule-guided mining.

5.5.2 Constraint Pushing: Mining Guided by Rule Constraints

Rule constraints specify expected set/subset relationships of the variables in the mined rules, constant initiation of variables, and aggregate functions. Users typically employ

their knowledge of the application or data to specify rule constraints for the mining task. These rule constraints may be used together with, or as an alternative to, metarule-guided mining. In this section, we examine rule constraints as to how they can be used to make the mining process more efficient. Let's study an example where rule constraints are used to mine hybrid-dimensional association rules.

Example 5.14 A closer look at mining guided by rule constraints. Suppose that *AllElectronics* has a sales multidimensional database with the following interrelated relations:

- *sales(customer_name, item_name, TID)*
- *lives_in(customer_name, region, city)*
- *item(item_name, group, price)*
- *transaction(TID, day, month, year)*

where *lives_in*, *item*, and *transaction* are three dimension tables, linked to the fact table *sales* via three keys, *customer_name*, *item_name*, and *TID* (*transaction_id*), respectively.

Our association mining query is to *"Find the sales of which cheap items (where the sum of the prices is less than $100) may promote the sales of which expensive items (where the minimum price is $500) of the same group for Chicago customers in 2004."* This can be expressed in the DMQL data mining query language as follows, where each line of the query has been enumerated to aid in our discussion:

(1) mine associations as
(2) $lives_in(C, _, \text{"}Chicago\text{"}) \wedge sales^{+}(C, ?\{I\}, \{S\}) \Rightarrow sales^{+}(C, ?\{J\}, \{T\})$
(3) from sales
(4) where S.year = 2004 and T.year = 2004 and I.group = J.group
(5) group by C, I.group
(6) having sum(I.price) $<$ 100 and min(J.price) $\geq$ 500
(7) with support threshold = 1%
(8) with confidence threshold = 50%

Before we discuss the rule constraints, let's look at the above query. Line 1 is a knowledge type constraint, where association patterns are to be discovered. Line 2 specifies a metarule. This is an abbreviated form for the following metarule for hybrid-dimensional association rules (multidimensional association rules where the repeated predicate here is *sales*):

$$lives_in(C, _, \text{"}Chicago\text{"}) \wedge sales(C, ?I_1, S_1) \wedge \ldots \wedge sales(C, ?I_k, S_k) \wedge I = \{I_1, \ldots, I_k\} \wedge S = \{S_1, \ldots, S_k\} \Rightarrow sales(C, ?J_1, T_1) \wedge \ldots \wedge sales(C, ?J_m, T_m) \wedge J = \{J_1, \ldots, J_m\} \wedge T = \{T_1, \ldots, T_m\}$$

which means that one or more *sales* records in the form of "$sales(C, ?I_1, S_1) \wedge \ldots sales(C, ?I_k, S_k)$" will reside at the rule antecedent (left-hand side), and the question mark "?"

means that only *item_name*, $I_1, \ldots, I_k$ need be printed out. "$I = \{I_1, \ldots, I_k\}$" means that all the Is at the antecedent are taken from a set I, obtained from the SQL-like **where** clause of line 4. Similar notational conventions are used at the consequent (right-hand side).

The metarule may allow the generation of association rules like the following:

$$\begin{aligned} &\mathit{lives_in}(C, _, \text{"}\mathit{Chicago}\text{"}) \wedge \mathit{sales}(C, \text{"}\mathit{Census_CD}\text{"}, _) \wedge \\ &\quad \mathit{sales}(C, \text{"}\mathit{MS/Office}\text{"}, _) \Rightarrow \mathit{sales}(C, \text{"}\mathit{MS/SQLServer}\text{"}, _) \quad [1.5\%, 68\%], \end{aligned} \tag{5.29}$$

which means that if a customer in Chicago bought "Census_CD" and "MS/Office," it is likely (with a probability of 68%) that the customer also bought "MS/SQLServer," and 1.5% of all of the customers bought all three.

Data constraints are specified in the "*lives_in*(C, _, "*Chicago*")" portion of the metarule (i.e., all the customers who live in Chicago) and in line 3, which specifies that only the fact table, *sales*, need be explicitly referenced. In such a multidimensional database, variable reference is simplified. For example, "*S.year = 2004*" is equivalent to the SQL statement "**from** *sales S, transaction T* **where** *S.TID = T.TID* **and** *T.year =2004*." All three dimensions (*lives_in, item*, and *transaction*) are used. Level constraints are as follows: for *lives_in*, we consider just *customer_name* since *region* is not referenced and *city = "Chicago"* is only used in the selection; for *item*, we consider the levels *item_name* and *group* since they are used in the query; and for *transaction*, we are only concerned with *TID* since *day* and *month* are not referenced and *year* is used only in the selection.

Rule constraints include most portions of the **where** (line 4) and **having** (line 6) clauses, such as "*S.year = 2004,*" "*T.year = 2004,*" "*I.group = J.group,*" "*sum(I.price)* $\leq$ *100,*" and "*min(J.price)* $\geq$ *500.*" Finally, lines 7 and 8 specify two interestingness constraints (i.e., thresholds), namely, a minimum support of 1% and a minimum confidence of 50%. ■

Dimension/level constraints and interestingness constraints can be applied after mining to filter out discovered rules, although it is generally more efficient and less expensive to use them *during* mining, to help prune the search space. Dimension/level constraints were discussed in Section 5.3, and interestingness constraints have been discussed throughout this chapter. Let's focus now on rule constraints.

"How can we use rule constraints to prune the search space? More specifically, what kind of rule constraints can be 'pushed' deep into the mining process and still ensure the completeness of the answer returned for a mining query?"

Rule constraints can be classified into the following five categories with respect to frequent itemset mining: (1) *antimonotonic,* (2) *monotonic,* (3) *succinct,* (4) *convertible,* and (5) *inconvertible.* For each category, we will use an example to show its characteristics and explain how such kinds of constraints can be used in the mining process.

The first category of constraints is **antimonotonic**. Consider the rule constraint "$\mathit{sum}(I.\mathit{price}) \leq 100$" of Example 5.14. Suppose we are using the Apriori framework, which at each iteration k explores itemsets of size k. If the price summation of the items

in an itemset is no less than 100, this itemset can be pruned from the search space, since adding more items into the set will only make it more expensive and thus will never satisfy the constraint. In other words, if an itemset does not satisfy this rule constraint, none of its supersets can satisfy the constraint. If a rule constraint obeys this property, it is **antimonotonic.** Pruning by antimonotonic constraints can be applied at each iteration of Apriori-style algorithms to help improve the efficiency of the overall mining process while guaranteeing completeness of the data mining task.

The Apriori property, which states that all nonempty subsets of a frequent itemset must also be frequent, is antimonotonic. If a given itemset does not satisfy minimum support, none of its supersets can. This property is used at each iteration of the Apriori algorithm to reduce the number of candidate itemsets examined, thereby reducing the search space for association rules.

Other examples of antimonotonic constraints include "$min(J.price) \geq 500$," "$count(I) \leq 10$," and so on. Any itemset that violates either of these constraints can be discarded since adding more items to such itemsets can never satisfy the constraints. Note that a constraint such as "$avg(I.price) \leq 100$" is not antimonotonic. For a given itemset that does not satisfy this constraint, a superset created by adding some (cheap) items may result in satisfying the constraint. Hence, pushing this constraint inside the mining process will not guarantee completeness of the data mining task. A list of SQL-primitives-based constraints is given in the first column of Table 5.12. The antimonotonicity of the constraints is indicated in the second column of the table. To simplify our discussion, only existence operators (e.g., $=$, $\in$, but not $\neq$, $\notin$) and comparison (or containment) operators with equality (e.g., $\leq$, $\subseteq$) are given.

The second category of constraints is **monotonic.** If the rule constraint in Example 5.14 were "$sum(I.price) \geq 100$," the constraint-based processing method would be quite different. If an itemset I satisfies the constraint, that is, the sum of the prices in the set is no less than 100, further addition of more items to I will increase cost and will always satisfy the constraint. Therefore, further testing of this constraint on itemset I becomes redundant. In other words, if an itemset satisfies this rule constraint, so do all of its supersets. If a rule constraint obeys this property, it is **monotonic.** Similar rule monotonic constraints include "$min(I.price) \leq 10$," "$count(I) \geq 10$," and so on. The monotonicity of the list of SQL-primitives-based constraints is indicated in the third column of Table 5.12.

The third category is **succinct constraints.** For this category of constraints, we can *enumerate all and only those sets that are guaranteed to satisfy the constraint.* That is, if a rule constraint is **succinct**, we can directly generate precisely the sets that satisfy it, even before support counting begins. This avoids the substantial overhead of the generate-and-test paradigm. In other words, such constraints are *precounting prunable.* For example, the constraint "$min(J.price) \geq 500$" in Example 5.14 is succinct, because we can explicitly and precisely generate all the sets of items satisfying the constraint. Specifically, such a set must contain at least one item whose price is no less than \$500. It is of the form $S_1 \cup S_2$, where $S_1 \neq \emptyset$ is a subset of the set of all those items with prices no less than \$500, and S_2, possibly empty, is a subset of the set of all those items with prices no greater than \$500. Because there is a precise "formula" for generating all of the sets satisfying a succinct constraint, there is no need to

Table 5.12 Characterization of commonly used SQL-based constraints.

Constraint	*Antimonotonic*	*Monotonic*	*Succinct*
$v \in S$	no	yes	yes
$S \supseteq V$	no	yes	yes
$S \subseteq V$	yes	no	yes
$min(S) \leq v$	no	yes	yes
$min(S) \geq v$	yes	no	yes
$max(S) \leq v$	yes	no	yes
$max(S) \geq v$	no	yes	yes
$count(S) \leq v$	yes	no	weakly
$count(S) \geq v$	no	yes	weakly
$sum(S) \leq v\ (\forall a \in S, a \geq 0)$	yes	no	no
$sum(S) \geq v\ (\forall a \in S, a \geq 0)$	no	yes	no
$range(S) \leq v$	yes	no	no
$range(S) \geq v$	no	yes	no
$avg(S)\ \theta\ v, \theta \in \{\leq, \geq\}$	convertible	convertible	no
$support(S) \geq \xi$	yes	no	no
$support(S) \leq \xi$	no	yes	no
$all_confidence(S) \geq \xi$	yes	no	no
$all_confidence(S) \leq \xi$	no	yes	no

iteratively check the rule constraint during the mining process. The succinctness of the list of SQL-primitives-based constraints is indicated in the fourth column of Table 5.12.[10]

The fourth category is **convertible constraints.** Some constraints belong to none of the above three categories. However, if the items in the itemset are arranged in a particular order, the constraint may become monotonic or antimonotonic with regard to the frequent itemset mining process. For example, the constraint "$avg(I.price) \leq 100$" is neither antimonotonic nor monotonic. However, if items in a transaction are added to an itemset in price-ascending order, the constraint becomes *antimonotonic*, because if an itemset I violates the constraint (i.e., with an average price greater than \$100), then further addition of more expensive items into the itemset will never make it satisfy the constraint. Similarly, if items in a transaction are added to an itemset in price-descending order, it becomes *monotonic*, because if the itemset satisfies the constraint (i.e., with an average price no

[10]For constraint $count(S) \leq v$ (and similarly for $count(S) \geq v$), we can have a member generation function based on a cardinality constraint (i.e., $\{X \mid X \subseteq Itemset \wedge |X| \leq v\}$). Member generation in this manner takes a different flavor and thus is called *weakly succinct*.

greater than \$100), then adding cheaper items into the current itemset will still make the average price no greater than \$100. Aside from "$avg(S) \leq v$," and "$avg(S) \geq v$," given in Table 5.12, there are many other convertible constraints, such as "$variance(S) \geq v$," "$standard_deviation(S) \geq v$," and so on.

Note that the above discussion does not imply that every constraint is convertible. For example, "$sum(S)\, \theta v$," where $\theta \in \{\leq, \geq\}$ and each element in S could be of any real value, is not convertible. Therefore, there is yet a fifth category of constraints, called **inconvertible constraints.** The good news is that although there still exist some tough constraints that are not convertible, most simple SQL expressions with built-in SQL aggregates belong to one of the first four categories to which efficient constraint mining methods can be applied.

5.6 Summary

- The discovery of frequent patterns, association, and correlation relationships among huge amounts of data is useful in selective marketing, decision analysis, and business management. A popular area of application is **market basket analysis**, which studies the buying habits of customers by searching for sets of items that are frequently purchased together (or in sequence). **Association rule mining** consists of first finding **frequent itemsets** (set of items, such as A and B, satisfying a *minimum support threshold*, or percentage of the task-relevant tuples), from which **strong** association rules in the form of $A \Rightarrow B$ are generated. These rules also satisfy a *minimum confidence threshold* (a prespecified probability of satisfying B under the condition that A is satisfied). Associations can be further analyzed to uncover **correlation rules**, which convey statistical correlations between itemsets A and B.
- **Frequent pattern mining** can be categorized in many different ways according to various criteria, such as the following:

  1. Based on the **completeness** of patterns to be mined, categories of frequent pattern mining include mining the *complete set of frequent itemsets*, the *closed frequent itemsets*, the *maximal frequent itemsets*, and *constrained frequent itemsets*.
  2. Based on the **levels** and **dimensions** of data involved in the rule, categories can include the mining of *single-level association rules*, *multilevel association rules*, *single-dimensional association rules*, and *multidimensional association rules*.
  3. Based on the **types of values** handled in the rule, the categories can include mining *Boolean association rules* and *quantitative association rules*.
  4. Based on the **kinds of rules** to be mined, categories include mining *association rules* and *correlation rules*.
  5. Based on the **kinds of patterns** to be mined, frequent pattern mining can be classified into *frequent itemset mining*, *sequential pattern mining*, *structured pattern mining*, and so on. This chapter has focused on frequent itemset mining.